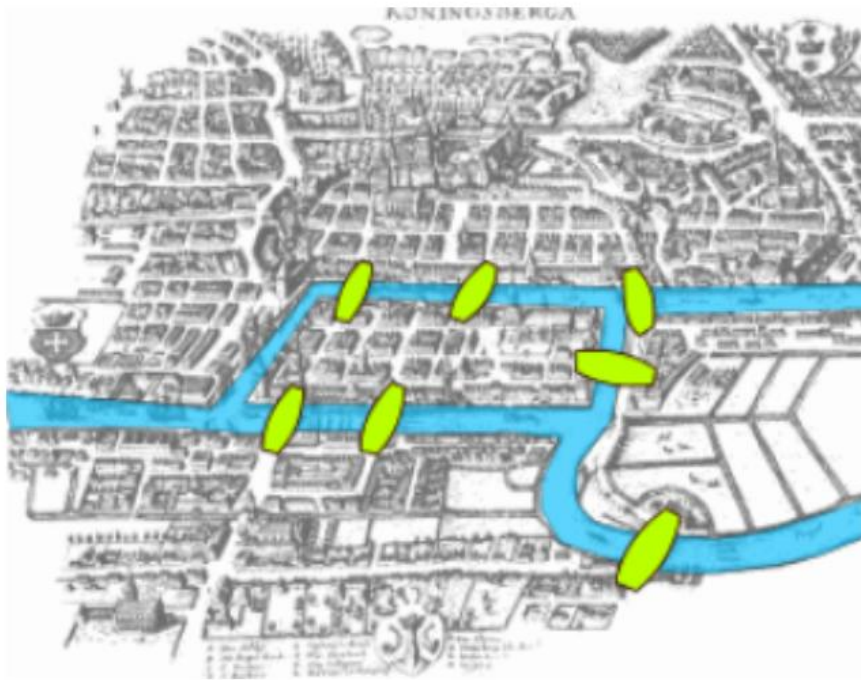


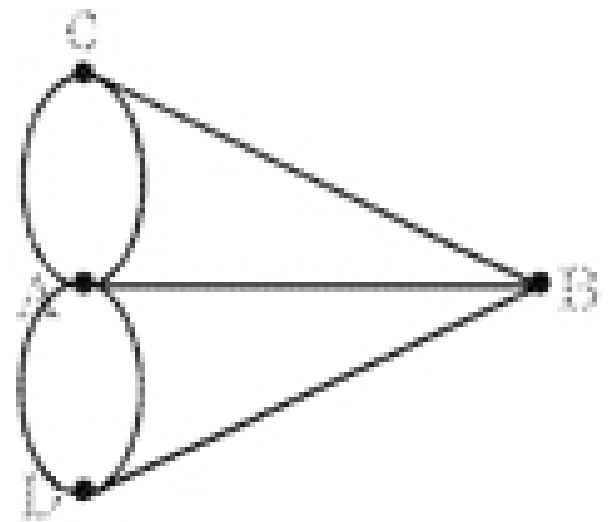
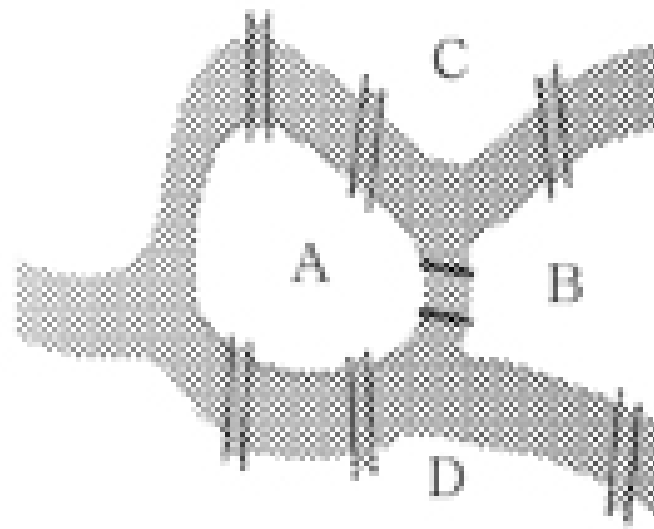
# LangGraph 개념 및 특징

# LangGraph의 탄생 배경

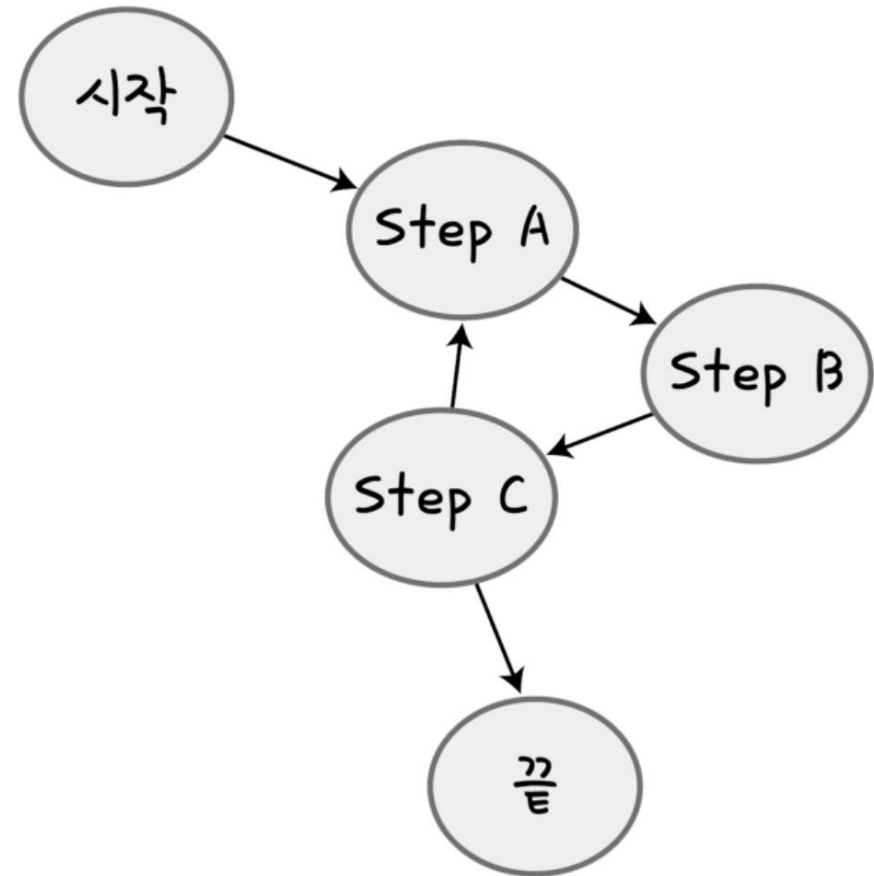
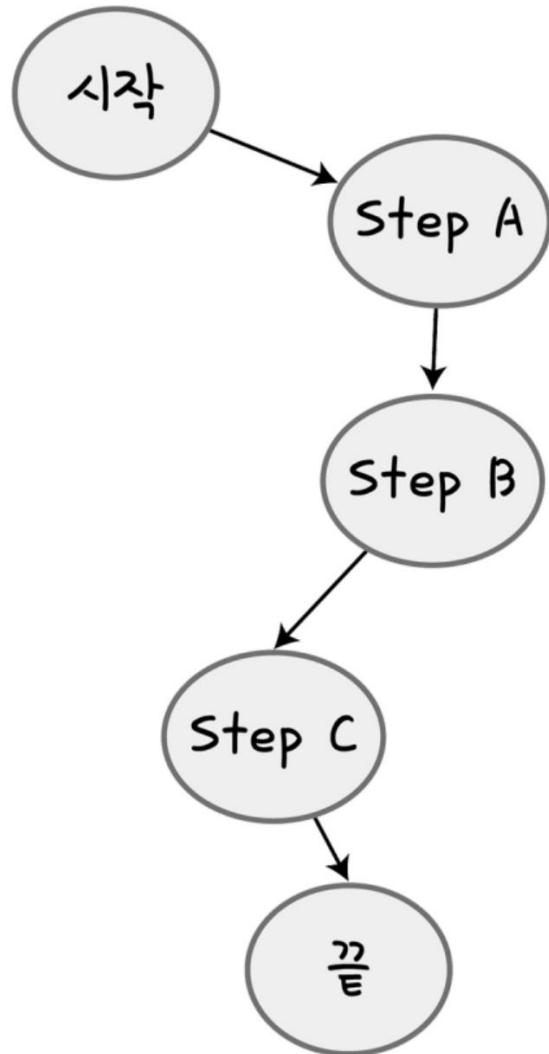
# 그래프



쾨니히스베르크 시와 일곱 개의 다리



# 그래프 구조



# 기존 RAG(Retrieval-Augmented Generation)의 문제점

- 단방향 파이프라인: 한 번에 모든 단계를 완벽하게 수행해야 함
- 피드백 불가능: 중간 단계에서 문제가 발생해도 되돌아가서 수정할 수 없음
- 고정된 구조:
  - 사전 정의된 데이터 로더
  - 고정된 청크 사이즈
  - 고정된 쿼리 입력
  - 고정된 프롬프트
  - LLM 답변 결과에 대한 문서와의 관련성/ 신뢰성

# 실제 문제 상황 예시

- 질문: "생성형 AI 가우스를 만든 회사의 2023년도 매출액을 알려줘"
- 문제 발생:
  1. 문서에서 "가우스를 만든 회사는 삼성전자"라는 정보는 찾음
  2. 하지만 삼성전자의 2023년도 매출액 정보는 문서에 없음
  3. 추가 검색이 필요하지만 기존 RAG는 유연하게 대응 불가
- 기존 해결 방식의 한계:
  - 체인을 덕지덕지 연결하여 복잡한 구조 생성
  - 잘못된 검색 결과로 인한 할루시네이션 발생 가능
  - 구조가 복잡해질수록 관리 및 디버깅 어려움

# LangGraph의 핵심 개념

- LangGraph는 기존 RAG의 한계를 극복하고, 더욱 유연하고 지능적인 정보 검색 및 생성 시스템을 구축할 수 있게 해주는 프레임워크
- 노드와 엣지 기반의 그래프 구조를 통해 복잡한 워크플로우를 체계적으로 관리하고, 동적인 처리를 통해 더 나은 결과를 얻을 수 있음

- **노드(Node)**

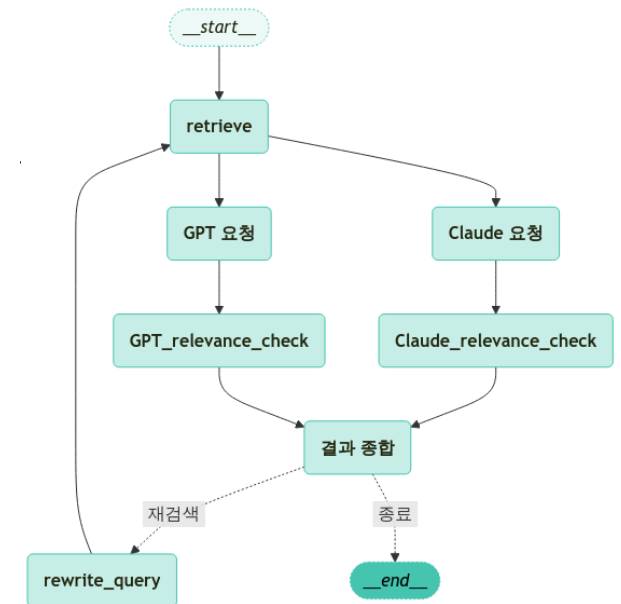
- 각각의 세부 과정을 독립적인 노드로 정의
- 예시: 로더 노드, 스플릿 노드, 임베딩 노드, 검색 노드, 프롬프트 노드, LLM 노드

- **엣지(Edge)**

- 노드와 노드를 연결하는 연결선
- 실행 흐름을 정의하는 역할

- **조건부 엣지(Conditional Edge)**

- 특정 조건에 따라 분기 처리 가능
- 다양한 분기 지원
- 라우팅 기법 구현 용이



# LangGraph의 주요 특징

- **플로우 엔지니어링**

- 노드들을 유연하게 배치하고 연결
- 실행 흐름을 동적으로 설계 가능
- 단방향이 아닌 다방향 흐름 구성

- **동적 처리 능력**

- **피드백 루프**: 결과가 만족스럽지 않으면 이전 단계로 돌아가서 재처리
- **사이클릭 구조**: 뱅글뱅글 돌면서 정보 보강 및 답변 개선
- **실시간 분기**: 실행 중 조건에 따라 다른 경로로 이동



# 예시 1: 관련성 평가를 통한 품질 향상

질문 입력 → 문서 검색 → 관련성 평가자

↓

[관련성 있음] → 답변 생성 → 최종 답변

↓

[관련성 없음] → 쿼리 재작성 → 문서 검색 (반복)

## 예시 2: 웹 검색을 통한 정보 보강

질문 입력 → 문서 검색 → 답변 생성 → 할루시네이션 체크

↓

[정보 부족] → 웹 검색 → 문서 보강 → 답변 재생성

↓

[품질 통과] → 최종 답변

# 예시 3: 복합 평가자 시스템

질문 입력 → 문서 검색 → 1차 평가자 → [웹 검색 필요/쿼리  
재작성/통과]

↓

답변 생성 → 2차 평가자 → 최종 답변

# LangGraph vs 기존 RAG 비교

| 구분<br>구조<br>피드백<br>분기 처리<br>오류 수정<br>복잡도 관리 | 기존 RAG                                            | LangGraph                                       |
|---------------------------------------------|---------------------------------------------------|-------------------------------------------------|
|                                             | 단방향, 고정적<br>불가능<br>어려움<br>한 번에 완벽해야 함<br>체인이 덕지덕지 | 다방향, 동적<br>가능<br>쉬움<br>반복 개선 가능<br>노드 기반 체계적 관리 |

# LangGraph의 장점

- **모듈러 RAG 구현**

- 각 기능을 독립적인 모듈(노드)로 분리
- 재사용성과 유지보수성 향상
- 모듈별 테스트 및 개선 가능

- **정교한 파이프라인 설계**

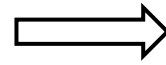
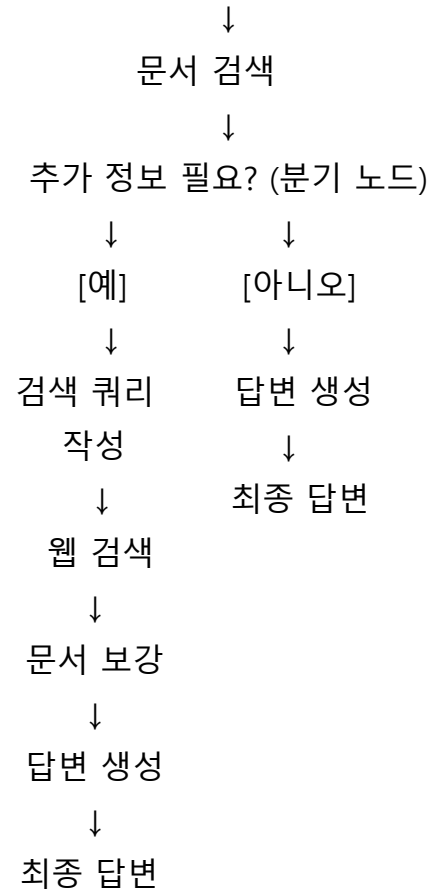
- 복잡한 비즈니스 로직을 직관적으로 표현
- 조건부 분기를 통한 지능적 처리
- 실행 흐름의 시각화 가능

- **품질 향상**

- 중간 검증 단계 추가 가능
- 반복적 개선을 통한 답변 품질 향상
- 할루시네이션 방지 메커니즘 구축

# 실제 구현 예시: 매출액 정보 검색

"생성형 AI 가우스를 만든 회사의 2023년도 매출액을 알려줘"



이러한 구조를 통해 문서에 없는 정보(삼성전자 매출액)를 웹에서 검색하여 보완하고, 완전한 답변을 생성할 수 있음

# LangGraph 에 자주 등장하는 Python 문법

# TypedDict

- dict와 TypedDict의 주요 차이점

- 타입 검사
  - dict: 런타임에 타입 검사를 하지 않습니다.
  - TypedDict: 정적 타입 검사를 제공합니다.
- 키와 값의 타입
  - dict: 키와 값의 타입을 일반적으로 지정합니다 (예: Dict[str, str]).
  - TypedDict: 각 키에 대해 구체적인 타입을 지정할 수 있습니다.
- 유연성
  - dict: 런타임에 키를 추가하거나 제거할 수 있습니다.
  - TypedDict: 정의된 구조를 따라야 합니다. 추가적인 키는 타입 오류를 발생시킵니다.

- TypedDict가 dict 대신 사용되는 이유

- 타입 안정성
  - TypedDict는 더 엄격한 타입 검사를 제공하여 잠재적인 버그를 미리 방지할 수 있습니다.
- 코드 가독성
  - TypedDict를 사용하면 딕셔너리의 구조를 명확하게 정의할 수 있어 코드의 가독성이 향상됩니다.
- IDE 지원
  - TypedDict를 사용하면 IDE에서 자동 완성 및 타입 힌트를 더 정확하게 제공할 수 있습니다.
- 문서화
  - TypedDict는 코드 자체가 문서의 역할을 하여 딕셔너리의 구조를 명확히 보여줍니다.



# TypedDict와 Dict의 차이점 예시

```
from typing import Dict, TypedDict
```

```
# 일반적인 파이썬 딕셔너리(dict) 사용
```

```
sample_dict: Dict[str, str] = {  
    "name": "은연",  
    "age": "20", # 문자열로 저장 (dict에서는 가능)  
    "job": "학생",  
}
```

```
# TypedDict 사용
```

```
class Person(TypedDict):  
    name: str  
    age: int # 정수형으로 명시  
    job: str
```

```
typed_dict: Person = {"name": "설리", "age": 25, "job": "디자이너"}
```

```
# dict의 경우
```

```
sample_dict["age"] = 35 # 문자열에서 정수로 변경되어도 오류 없음
```

```
sample_dict["new_field"] = "추가 정보" # 새로운 필드 추가 가능
```

```
# TypedDict의 경우
```

```
typed_dict["age"] = 35 # 정수형으로 올바르게 사용
```

```
typed_dict["age"] = "35" # 타입 체커가 오류를 감지함
```

```
typed_dict["new_field"] = (  
    "추가 정보" # 타입 체커가 정의되지 않은 키라고 오류를 발생시  
    킴  
)
```

# Annotated

- 타입 힌트에 메타데이터를 추가할 수 있게 해줌=>추가 정보 제공(타입 힌트) / 문서화
- 타입 힌트에 추가적인 정보를 포함시킬 수 있음=>코드를 읽는 사람이나 도구에 더 많은 컨텍스트를 제공
- 코드에 대한 추가 설명을 타입 힌트에 직접 포함시킬 수 있음
  - name: Annotated[str, "이름"]
  - age: Annotated[int, "나이"]
- Annotated 는 Python의 typing 모듈에서 제공하는 특별한 타입 힌트로, 기존 타입에 메타데이터를 추가할 수 있게 해줌=>이를 통해 코드의 가독성을 높이고, 더 자세한 타입 정보를 제공할 수 있음

# Annotated 주요 기능(사용 이유)

- 추가 정보 제공: 타입 힌트에 메타데이터를 추가하여 더 상세한 정보를 제공
- 문서화: 코드 자체에 추가 설명을 포함시켜 문서화 효과를 얻을 수 있음
- 유효성 검사: 특정 라이브러리(예: Pydantic)와 함께 사용하여 데이터 유효성 검사를 수행할 수 있음
- 프레임워크 지원: 일부 프레임워크(예: LangGraph)에서는 Annotated를 사용하여 특별한 동작을 정의
- 기본 문법
  - Type: 기본 타입 (예: int, str, List[str] 등)
  - metadata1, metadata2, ...: 추가하고자 하는 메타데이터

```
from typing import Annotated
```

```
variable: Annotated[Type, metadata1, metadata2, ...]
```

# Annotated 사용 예시

- 기본 사용

```
from typing import Annotated
name: Annotated[str, "사용자 이름"]
age: Annotated[int, "사용자 나이 (0-150)"]
```

- LangGraph에서의 사용(add\_messages)

```
from typing import Annotated, TypedDict
from langgraph.graph import add_messages
class MyData(TypedDict):
    messages: Annotated[list, add_messages]
```

- 참고

Annotated는 Python 3.9 이상에서 사용 가능합니다.  
런타임에는 Annotated가 무시되므로, 실제 동작에는 영향을 주지 않습니다.  
타입 검사 도구나 IDE가 Annotated를 지원해야 그 효과를 볼 수 있습니다.

# add\_messages: 리뷰서 함수

- 함수는 2개의 인자(left, right)를 받으며 좌, 우 메시지를 병합하는 방식으로 동작
- 주요 기능
  - 두 개의 메시지 리스트를 병합
  - 기본적으로 "append-only" 상태를 유지
  - 동일한 ID를 가진 메시지가 있을 경우, 새 메시지로 기존 메시지를 대체
- 동작 방식
  - right의 메시지 중 left에 동일한 ID를 가진 메시지가 있으면, right의 메시지로 대체
  - 그 외의 경우 right의 메시지가 left에 추가
- 매개변수
  - left (Messages): 기본 메시지 리스트
  - right (Messages): 병합할 메시지 리스트 또는 단일 메시지
- 반환값
  - Messages: right의 메시지들이 left에 병합된 새로운 메시지 리스트

```
from langchain_core.messages import AIMessage, HumanMessage
from langgraph.graph import add_messages

# 기본 사용 예시
msgs1 = [HumanMessage(content="안녕하세요?", id="1")]
msgs2 = [AIMessage(content="반갑습니다~", id="2")]

result1 = add_messages(msgs1, msgs2)
print(result1)
```

# 리듀서 함수란?

- LangGraph에서 상태(state)가 업데이트될 때 새로운 값과 기존 값을 어떻게 결합할지를 정의하는 함수

```
from typing import Annotated
from langgraph.graph import add_messages
# messages 키에 add_messages 리듀서를 주석으로 적용
class State(TypedDict):
    messages: Annotated[list, add_messages] # 리듀서 함수로 주석 처리
    user_info: str # 주석 없음 - 덮어쓰기 방식
# 초기 상태state = {"messages": []}
# 첫 번째 업데이트
update1 = {"messages": [{"role": "user", "content": "안녕하세요"}]}
# 두 번째 업데이트
update2 = {"messages": [{"role": "assistant", "content": "안녕하세요! 무엇을 도와드릴까요?"}]}
# 세 번째 업데이트
update3 = {"messages": [{"role": "user", "content": "날씨가 어때요?"}]}
```

```
# 초기 상태
state = {"user_info": ""}
# 첫 번째 업데이트
update1 = {"user_info": "홍길동"}
# 결과: {"user_info": "홍길동"}
# 두 번째 업데이트
update2 = {"user_info": "김철수"}
# 결과: {"user_info": "김철수"} # 이전 값이 완전히 대체됨
```

# 실제 사용 예시

```
from langgraph.graph import StateGraph, add_messages
class ChatState(TypedDict):
    messages: Annotated[list, add_messages] # 대화 이력 누적
    current_topic: str # 현재 주제 (덮어쓰기)
    user_name: str # 사용자 이름 (덮어쓰기)

def chatbot_node(state: ChatState):
    # 새 메시지 추가 - 기존 메시지에 누적됨
    return {
        "messages": [{"role": "assistant", "content": "네, 도와드리겠습니다!"}],
        "current_topic": "일반_대화" # 이전 주제가 있다면 덮어쓰여짐
    }

def user_input_node(state: ChatState):
    return {
        "messages": [{"role": "user", "content": "도움이 필요해요"}],
        "user_name": "홍길동" # 이전 이름이 있다면 덮어쓰여짐
    }
```

# 리듀서 함수 주요 장점

- **메모리 관리**

- 대화 이력을 자동으로 누적하여 컨텍스트 유지
- 불필요한 정보는 덮어쓰기로 메모리 효율성 확보

- **코드 간소화**

- 수동으로 리스트를 합치는 로직 불필요
- 상태 업데이트 로직이 자동화됨

- **일관성 보장**

- 모든 노드에서 동일한 방식으로 메시지 처리
- 실수로 메시지가 누락되거나 중복되는 것 방지



# 랭그래프 기본 설계

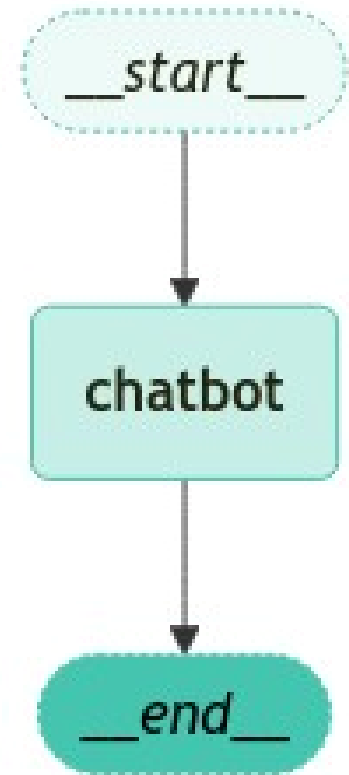
# STEP 1. 상태(State) 정의

```
from typing import Annotated, TypedDict

from langgraph.graph import StateGraph, START, END

from langgraph.graph.message import add_messages

class State(TypedDict):
    # 메시지 정의(list type 이며 add_messages 함수를 사용하여 메시지를 추가)
    messages: Annotated[list, add_messages]
```



# STEP 2. 노드(Node) 정의

```
from langchain_openai import ChatOpenAI
```

```
# LLM 정의
```

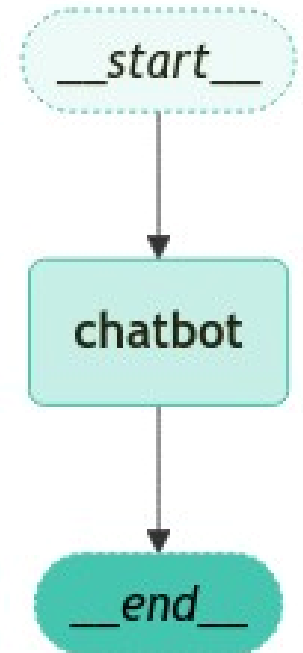
```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

```
# 챗봇 함수 정의
```

```
def chatbot(state: State):
```

```
    # 메시지 호출 및 반환
```

```
    return {"messages": [llm.invoke(state["messages"])]}
```



## STEP 3. 그래프(Graph) 정의, 노드 추가

# 그래프 생성

```
graph_builder = StateGraph(State)
```

# 노드 이름, 함수 혹은 callable 객체를 인자로 받아 노드를 추가

```
graph_builder.add_node("chatbot", chatbot)
```

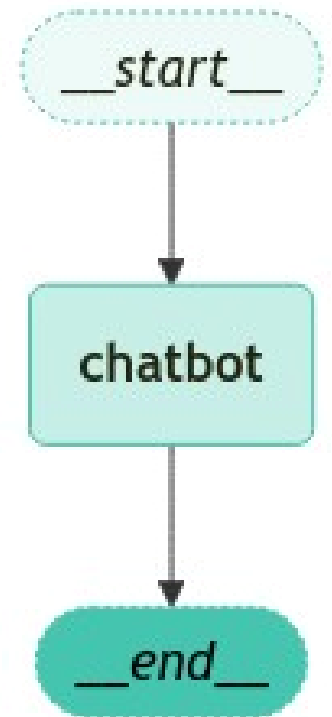
## STEP 4. 그래프 엣지(Edge) 추가

# 시작 노드에서 챗봇 노드로의 엣지 추가

```
graph_builder.add_edge(START, "chatbot")
```

# 그래프에 엣지 추가

```
graph_builder.add_edge("chatbot", END)
```



## STEP 5. 그래프 컴파일(compile)

```
# 그래프 컴파일
```

```
graph = graph_builder.compile()
```

## STEP 6. 그래프 시각화

```
from IPython.display import Image, display
```

```
try:
```

```
    display( Image(graph.get_graph().draw_mermaid_png()) ) # 그래프로 시각화
```

```
except Exception: # 예외 처리
```

```
    pass
```

## STEP 7. 그래프 실행

```
question = "서울의 유명한 맛집 TOP 10 추천해줘"
```

```
# 그래프 이벤트 스트리밍
```

```
for event in graph.stream({"messages": [("user", question)]}):
```

```
    # 이벤트 값 출력
```

```
    for value in event.values():
```

```
        print("Assistant:", value["messages"][-1].content)
```

